



Processador ARM: Implementação

Viviane Flor Park 169259



Sumário

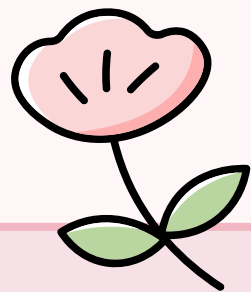
- Contextualização
 - Características do processador
- Datapaths: Antigo vs. Atual
- Implementações em Verilog
 - Módulos individuais e integração
- Código teste
 - Forma de onda

Contextualização

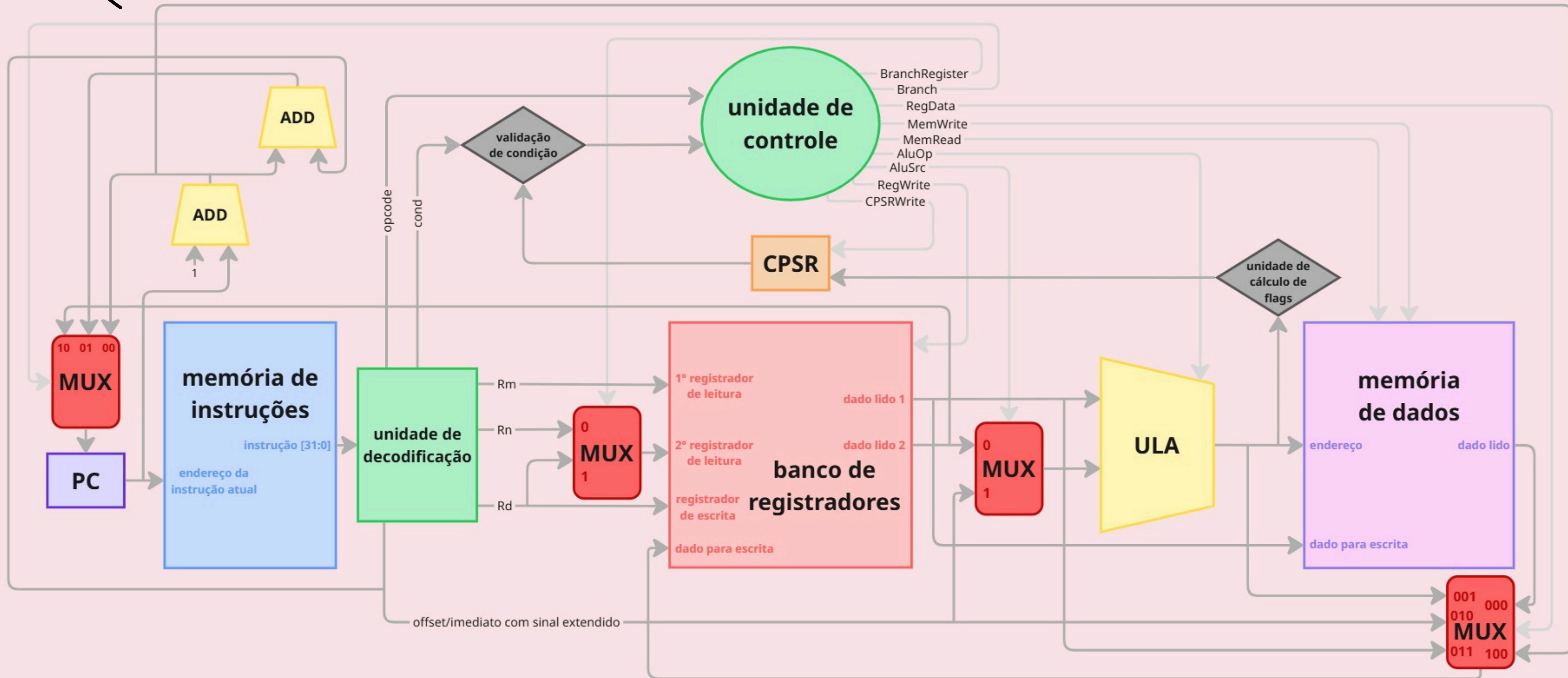
- Processador ARM 32 bits
 - Execução condicional de instruções
- 32 registradores de propósito geral
 - R31 = Link Register
- Current Program Status Register (CPSR)
- Program Counter (PC)

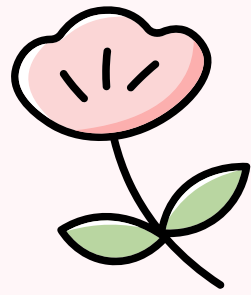
Contextualização

- Conjunto de Instruções
 - Instruções de Processamento de Dados (D)
 - Instruções de Desvio ou Branch (B)
 - Instruções de Acesso à Memória (M)
 - 4 Modos de Endereçamento
 - Instruções Alternativas (A)

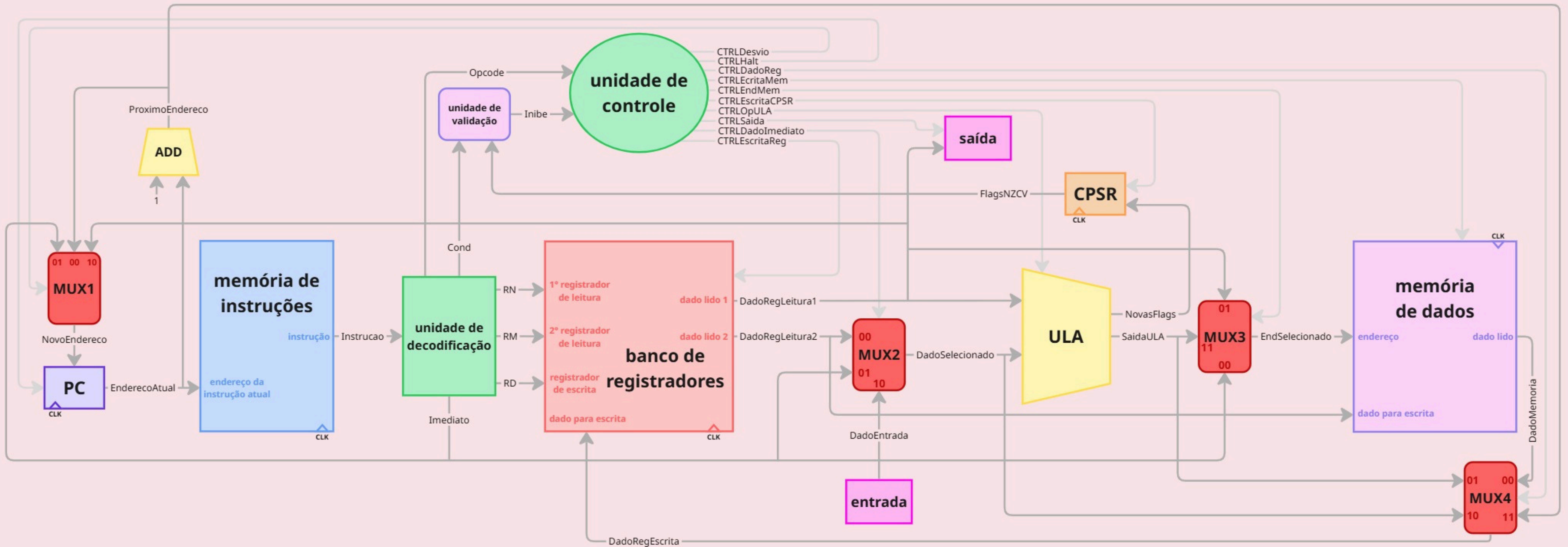


Datapath Antigo (PC1)





Datapath Novo





Implementação em Verilog: Módulos

Program Counter

```
1  module PC
2  (
3  // Entradas :: Endereço da Nova Instrução
4  input wire [31:0] NovoEndereco,
5
6  // Entradas :: Sinais de Controle
7  input wire CLK, CTRLHalt,
8
9  // Saídas :: Endereço da Instrução Atual
10 output reg [31:0] EnderecoAtual
11 );
12
13     always @ (posedge CLK)
14     begin
15         if (CTRLHalt == 1'b0)
16             begin
17                 EnderecoAtual <= NovoEndereco;
18             end
19     end
20
21 endmodule
```

- Atualizado com a borda de subida do clock
- CTRLHalt inibe sua atualização



Memória de Instruções

```
1 module MemInstrucoes
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=5)
3   ( // Entradas :: Endereço da Instrução Atual
4     input [(ADDR_WIDTH-1):0] EnderecoAtual,
5
6     // Entradas :: Sinais de Controle
7     input CLK,
8
9     // Saídas :: Instrução Atual
10    output reg [(DATA_WIDTH-1):0] Instrucao
11  );
12
13  // Variáveis Auxiliares :: Memória ROM
14  reg [DATA_WIDTH-1:0] rom[2**ADDR_WIDTH-1:0];
15
16  initial
17  begin
18    $readmemb("MemInstrDados.txt", rom);
19  end
20
21  always @ (negedge CLK)
22  begin
23    Instrucao <= rom[EnderecoAtual];
24  end
25
26 endmodule
```

- Leitura sequencial com a borda de descida do clock
- 32 posições

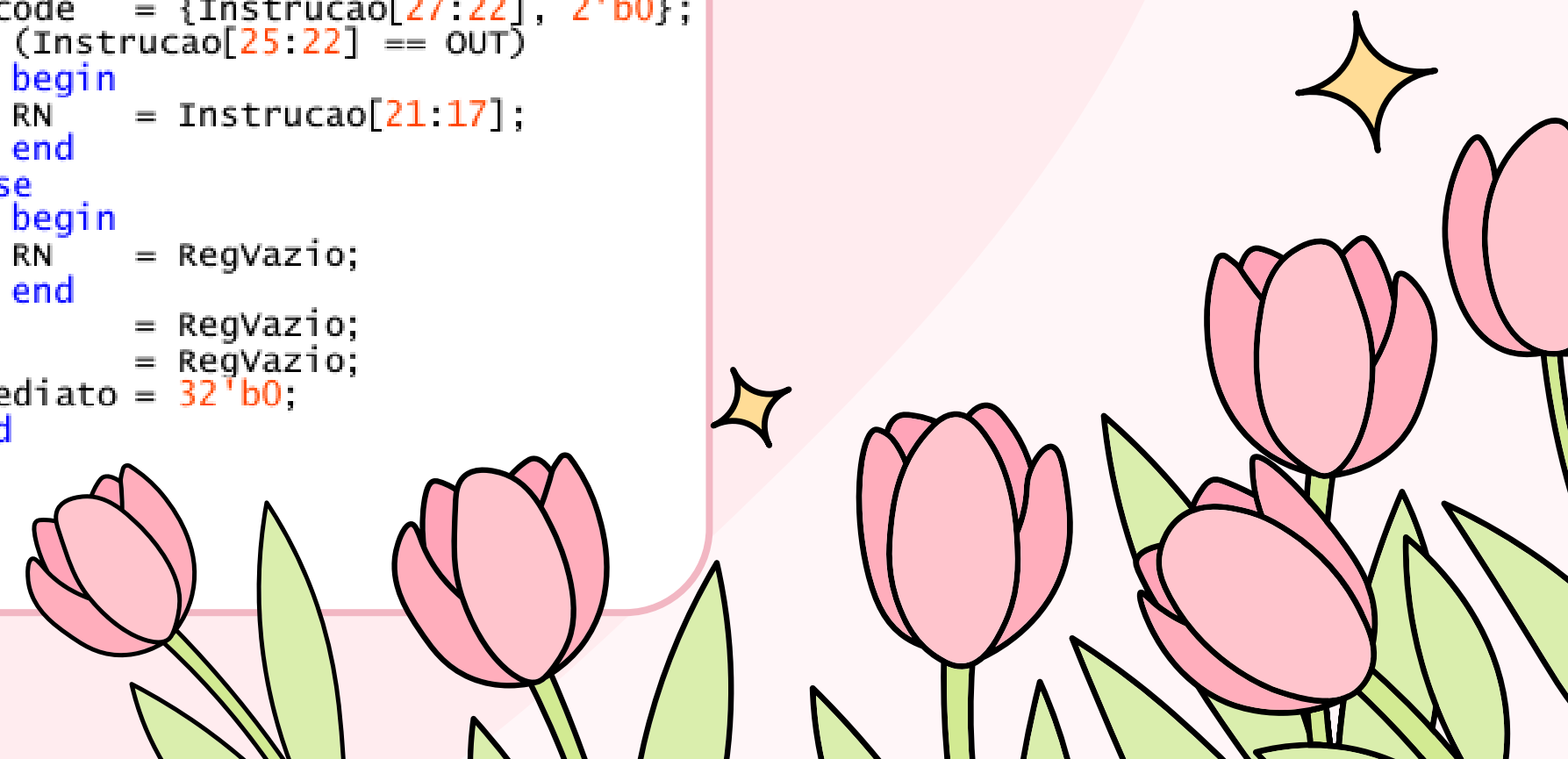


Unidade de Decodificação

```
1 module UnidadeDecodificacao
2 (
3 // Entradas :: Instrução Atual
4 input wire [31:0] Instrucao,
5
6 // Saídas :: Cond & Opcode
7 output reg [3:0] Cond,
8 output reg [7:0] Opcode,
9
10 // Saídas :: Registradores
11 output reg [4:0] RN, RD, RM,
12
13 // Saídas :: Offset/Imediato
14 output reg [31:0] Imediato
15 );
16
17 // Parâmetros :: Tipos de Dados
18 localparam TipoD = 2'b00, TipoB = 2'b01,
19             TipoM = 2'b10, TipoA = 2'b11;
20
21 // Outros Parâmetros
22 localparam RegVazio = 5'b0, LR = 5'b11111, BX = 2'b01,
23             BL = 2'b10, OUT = 4'b0011;
24
25 always @ (*)
26 begin
27     case (Instrucao[27:26])
28         TipoD:
29             begin
30                 Cond = Instrucao[31:28];
31                 Opcode = Instrucao[27:20];
32                 RN = Instrucao[19:15];
33                 RD = Instrucao[14:10];
34                 RM = Instrucao[9:5];
35                 Imediato = {22'b0, Instrucao[9:0]};
36             end
37         TipoB:
38             begin
39                 Cond = Instrucao[31:28];
40                 Opcode = {Instrucao[27:24], 4'b0};
41                 if (Instrucao[25:24] == BX)
42                     begin
43                         RN = Instrucao[23:19];
44                         RD = RegVazio;
45                     end
46                 if (Instrucao[25:24] == BL)
47                     begin
48                         RN = RegVazio;
49                         RD = LR;
50                     end
51                 RM = RegVazio;
52                 Imediato = {13'b0, Instrucao[18:0]};
53             end
```

```
54             TipoM:
55                 begin
56                     Cond = Instrucao[31:28];
57                     Opcode = {Instrucao[27:23], 3'b0};
58                     RN = Instrucao[22:18];
59                     if (Instrucao[25] == 1'b0)
60                         begin
61                             RD = Instrucao[17:13];
62                             RM = RegVazio;
63                         end
64                     if (Instrucao[25] == 1'b1)
65                         begin
66                             RD = RegVazio;
67                             RM = Instrucao[17:13];
68                         end
69                     Imediato = {19'b0, Instrucao[12:0]};
70                 end
71             TipoA:
72                 begin
73                     Cond = Instrucao[31:28];
74                     Opcode = {Instrucao[27:22], 2'b0};
75                     if (Instrucao[25:22] == OUT)
76                         begin
77                             RN = Instrucao[21:17];
78                         end
79                     else
80                         begin
81                             RN = RegVazio;
82                         end
83                     RD = RegVazio;
84                     RM = RegVazio;
85                     Imediato = 32'b0;
86                 end
87             endcase
88         end
89     endmodule
90
```

- Campo opo
- Extensão de sinal



Banco de Registradores

```
1  module BancoRegistradores
2  (
3  // Entradas :: Registradores de Leitura
4  input wire [4:0] RN, RM,
5
6  // Entradas :: Dados para Escrita
7  input wire [4:0] RD,
8  input wire [31:0] DadoRegEscrita,
9
10 // Entradas :: Sinais de Controle
11 input wire CTRLEscritaReg, CLK,
12
13 // Saídas :: Dados Lidos
14 output reg [31:0] DadoRegLeitura1, DadoRegLeitura2
15 );
16
17 // Variáveis Auxiliares :: Banco de Registradores
18 reg [31:0] Registradores[31:0];
19
20 always @ (*)
21 begin
22     DadoRegLeitura1 <= Registradores[RN];
23     DadoRegLeitura2 <= Registradores[RM];
24 end
25
26 always @ (posedge CLK)
27 begin
28     if (CTRLEscritaReg == 1'b1)
29     begin
30         Registradores[RD] <= DadoRegEscrita;
31     end
32 end
33
34 endmodule
```

- Escrita sequencial com a borda de subida do clock
- Leitura combinacional

Unidade Lógica e Aritmética

```
1 module ULA
2 (
3 // Entradas :: Dados
4 input wire [31:0] Dado1, Dado2,
5
6 // Entradas :: Sinais de Controle
7 input wire [3:0] CTRLopULA,
8
9 // Saídas
10 output wire [31:0] SaidaULA,
11 output wire [3:0] NovasFlags
12 );
13
14 // Variáveis e Parâmetros Auxiliares
15 reg [32:0] AuxiliarCV;
16 reg [31:0] SaidaAuxiliar;
17 reg [1:0] FlagsAuxiliar;
18 localparam Zero = 32'b0;
19
20 // Parâmetros :: CTRLopULA
21 localparam Soma = 4'b0000, Sub1 = 4'b0001, Sub2 = 4'b0010, Mul = 4'b0011,
22 Div = 4'b0100, Not = 4'b0101, And = 4'b0110, Or = 4'b0111, Xor =
4'b1000;
23
24 // Operações da ULA
25 always @ (CTRLopULA or Dado1 or Dado2)
26 begin
27 FlagsAuxiliar[1:0] = 2'b0;
28 case(CTRLopULA[3:0])
29 Soma: begin
30 AuxiliarCV = {1'b0, Dado1} + {1'b0, Dado2};
31 SaidaAuxiliar = AuxiliarCV[31:0];
32 FlagsAuxiliar[1] = AuxiliarCV[32];
33 FlagsAuxiliar[0] = (Dado1[31] == Dado2[31]) && (SaidaAuxiliar[31] != Dado1[
31]);
34 end
35 Sub1: begin
36 AuxiliarCV = {1'b0, Dado1} - {1'b0, Dado2};
37 SaidaAuxiliar = AuxiliarCV[31:0];
38 FlagsAuxiliar[1] = ~AuxiliarCV[32];
39 FlagsAuxiliar[0] = (Dado1[31] != Dado2[31]) && (SaidaAuxiliar[31] == Dado2[
31]);
40 end
41 Sub2: begin
42 AuxiliarCV = {1'b0, Dado2} - {1'b0, Dado1};
43 SaidaAuxiliar = AuxiliarCV[31:0];
44 FlagsAuxiliar[1] = ~AuxiliarCV[32];
45 FlagsAuxiliar[0] = (Dado1[31] != Dado2[31]) && (SaidaAuxiliar[31] == Dado1[
31]);
46 end
47 Mul: SaidaAuxiliar = Dado1 * Dado2;
48 Div: SaidaAuxiliar = Dado1 / Dado2;
49 Not: SaidaAuxiliar = ~Dado2;
50 And: SaidaAuxiliar = Dado1 & Dado2;
51 Or: SaidaAuxiliar = Dado1 | Dado2;
52 Xor: SaidaAuxiliar = Dado1 ^ Dado2;
53 default: SaidaAuxiliar = Zero;
54 endcase
55 end
56
57 // Atribuição de Valores às Saídas
58 assign SaidaULA = SaidaAuxiliar;
59 assign NovasFlags = {SaidaAuxiliar[31], (SaidaAuxiliar == Zero), FlagsAuxiliar[1:0]};
60
61 endmodule
```

- 9 operações

- Carry e overflow calculados apenas para soma e subtração

Current Program Status Register

```
1  module CPSR
2  (
3  // Entradas :: Novas Flags Calculadas
4  input wire [3:0] NovasFlags,
5
6  // Entradas :: Sinais de Controle
7  input wire CTRLEscritaCPSR,
8  input wire CLK,
9
10 // Saídas :: Flags NZCV
11 output reg [3:0] FlagsNZCV
12 );
13
14     always @ (posedge CLK)
15     begin
16         if (CTRLEscritaCPSR == 1'b1)
17             begin
18                 FlagsNZCV <= NovasFlags;
19             end
20     end
21
22 endmodule
```

- Atualizado com a borda de subida do clock



Memória de Dados

```
1 module MemPrincipal
2   #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=5)
3   (
4     // Entradas :: Endereços para Leitura e Escrita
5     input [(ADDR_WIDTH-1):0] EndLeitura, EndEscrita,
6
7     // Entradas :: Dado para Escrita
8     input [(DATA_WIDTH-1):0] DadoMemEscrita,
9
10    // Entradas :: Sinais de Controle
11    input CTRLEscritaMem, CLK,
12
13    // Saída :: Dado Lido
14    output reg [(DATA_WIDTH-1):0] DadoMemoria
15  );
16
17  // Variáveis Auxiliares :: Memória
18  reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];
19
20  always @ (*)
21  begin
22    DadoMemoria = ram[EndLeitura];
23  end
24
25  always @ (posedge CLK)
26  begin
27    if (CTRLEscritaMem == 1'b1)
28      ram[EndEscrita] <= DadoMemEscrita;
29  end
30 endmodule
```

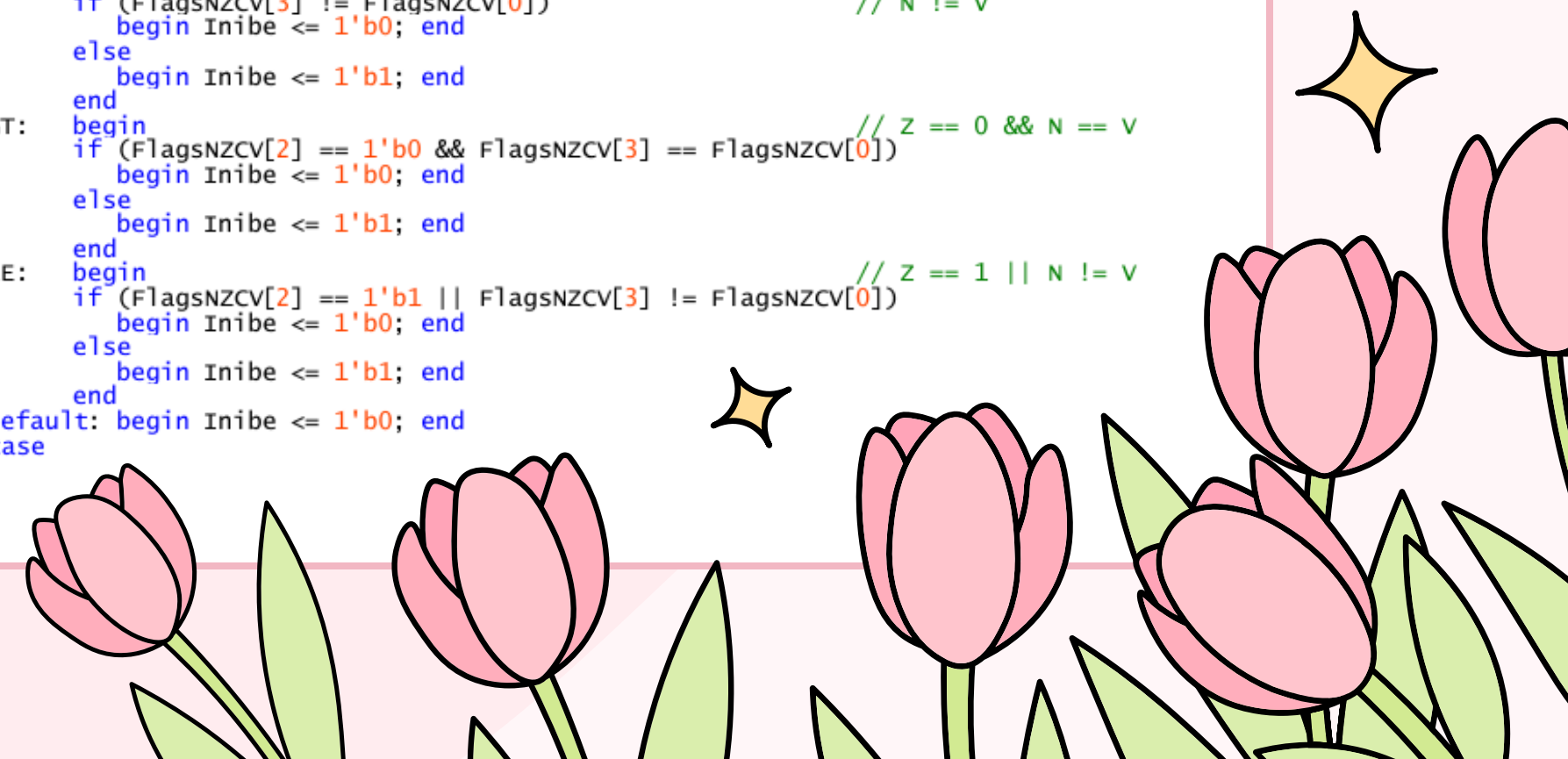
- Escrita sequencial com a borda de subida do clock
- Leitura combinacional
- 32 posições



Unidade de Validação

```
1 module UnidadeValidacao
2 (
3 // Entradas
4 input wire [3:0] Cond, FlagsNZCV,
5
6 // Saídas
7 output reg Inibe
8 );
9
10 localparam EQ = 4'b0000, NE = 4'b0001, CS = 4'b0010, CC = 4'b0011, MI = 4'b0100,
11            PL = 4'b0101, VS = 4'b0110, VC = 4'b0111, HI = 4'b1000, LS = 4'b1001,
12            GE = 4'b1010, LT = 4'b1011, GT = 4'b1100, LE = 4'b1101;
13
14 always @ (*)
15 begin
16     case(Cond[3:0])
17     EQ: begin
18         if (FlagsNZCV[2] == 1'b1) // Z == 1
19             begin Inibe <= 1'b0; end
20         else
21             begin Inibe <= 1'b1; end
22         end
23     NE: begin
24         if (FlagsNZCV[2] == 1'b0) // Z == 0
25             begin Inibe <= 1'b0; end
26         else
27             begin Inibe <= 1'b1; end
28         end
29     CS: begin
30         if (FlagsNZCV[1] == 1'b1) // C == 1
31             begin Inibe <= 1'b0; end
32         else
33             begin Inibe <= 1'b1; end
34         end
35     CC: begin
36         if (FlagsNZCV[1] == 1'b0) // C == 0
37             begin Inibe <= 1'b0; end
38         else
39             begin Inibe <= 1'b1; end
40         end
41     MI: begin
42         if (FlagsNZCV[3] == 1'b1) // N == 1
43             begin Inibe <= 1'b0; end
44         else
45             begin Inibe <= 1'b1; end
46         end
47     PL: begin
48         if (FlagsNZCV[3] == 1'b0) // N == 0
49             begin Inibe <= 1'b0; end
50         else
51             begin Inibe <= 1'b1; end
52         end
53     VS: begin
```

```
54         if (FlagsNZCV[0] == 1'b1) // V == 1
55             begin Inibe <= 1'b0; end
56         else
57             begin Inibe <= 1'b1; end
58         end
59     VC: begin
60         if (FlagsNZCV[0] == 1'b0) // V == 0
61             begin Inibe <= 1'b0; end
62         else
63             begin Inibe <= 1'b1; end
64         end
65     HI: begin
66         if (FlagsNZCV[1] == 1'b1 && FlagsNZCV[2] == 1'b0) // C == 1 && Z == 0
67             begin Inibe <= 1'b0; end
68         else
69             begin Inibe <= 1'b1; end
70         end
71     LS: begin
72         if (FlagsNZCV[1] == 1'b0 || FlagsNZCV[2] == 1'b1) // C == 0 || Z == 1
73             begin Inibe <= 1'b0; end
74         else
75             begin Inibe <= 1'b1; end
76         end
77     GE: begin
78         if (FlagsNZCV[3] == FlagsNZCV[0]) // N == V
79             begin Inibe <= 1'b0; end
80         else
81             begin Inibe <= 1'b1; end
82         end
83     LT: begin
84         if (FlagsNZCV[3] != FlagsNZCV[0]) // N != V
85             begin Inibe <= 1'b0; end
86         else
87             begin Inibe <= 1'b1; end
88         end
89     GT: begin
90         if (FlagsNZCV[2] == 1'b0 && FlagsNZCV[3] == FlagsNZCV[0]) // Z == 0 && N == V
91             begin Inibe <= 1'b0; end
92         else
93             begin Inibe <= 1'b1; end
94         end
95     LE: begin
96         if (FlagsNZCV[2] == 1'b1 || FlagsNZCV[3] != FlagsNZCV[0]) // Z == 1 || N != V
97             begin Inibe <= 1'b0; end
98         else
99             begin Inibe <= 1'b1; end
100        end
101    default: begin Inibe <= 1'b0; end
102    endcase
103 end
104
105 endmodule
```



cond	Mnemonic extension	Meaning (integer)	Meaning (floating-point) ^a	Condition flags
0000	EQ	Equal	Equal	$Z == 1$
0001	NE	Not equal	Not equal, or unordered	$Z == 0$
0010	CS ^b	Carry set	Greater than, equal, or unordered	$C == 1$
0011	CC ^c	Carry clear	Less than	$C == 0$
0100	MI	Minus, negative	Less than	$N == 1$
0101	PL	Plus, positive or zero	Greater than, equal, or unordered	$N == 0$
0110	VS	Overflow	Unordered	$V == 1$
0111	VC	No overflow	Not unordered	$V == 0$
1000	HI	Unsigned higher	Greater than, or unordered	$C == 1$ and $Z == 0$
1001	LS	Unsigned lower or same	Less than or equal	$C == 0$ or $Z == 1$
1010	GE	Signed greater than or equal	Greater than or equal	$N == V$
1011	LT	Signed less than	Less than, or unordered	$N != V$
1100	GT	Signed greater than	Greater than	$Z == 0$ and $N == V$
1101	LE	Signed less than or equal	Less than, equal, or unordered	$Z == 1$ or $N != V$
1110	None (AL) ^d	Always (unconditional)	Always (unconditional)	Any

Fonte: Manual de Referência ARMv8/v9-A. Disponível [neste link](#).

Unidade de Controle

```

1 module UnidadeControle
2 (
3 // Entradas :: Opcode & Inibe
4 input wire [7:0] Opcode,
5 input wire Inibe,
6
7 // Saídas :: Sinais de Controle :: MUX
8 output reg [1:0] CTRLDesvio, CTRLDadoImediato, CTRLEndMem, CTRLDadoReg,
9
10 // Saídas :: Sinais de Controle :: Escrita
11 output reg CTREscritaReg, CTREscritaCPSR, CTREscritaMem,
12
13 // Saídas :: Sinais de Controle :: ULA
14 output reg [3:0] CTRLopULA,
15
16 // Saídas :: Sinais de Controle :: Halt
17 output reg CTRLHalt
18 );
19
20 // Parâmetros Auxiliares :: Tipos de Instruções
21 localparam TipoD = 2'b00, TipoB = 2'b01, TipoM = 2'b10, TipoA = 2'b11;
22
23 // Parâmetros Auxiliares :: Instruções do Tipo D
24 localparam ADD = 5'b00000, ADDS = 5'b00001, SUB = 5'b00010, SUBS = 5'b00011,
25 RSB = 5'b00100, MUL = 5'b00101, UDIV = 5'b00110, NOT = 5'b00111,
26 AND = 5'b01000, ORR = 5'b01001, EOR = 5'b01010, MOV = 5'b01011,
27 CMP = 5'b01100;
28
29 // Parâmetros Auxiliares :: Instruções do Tipo B
30 localparam B = 2'b00, BX = 2'b01, BL = 2'b10;
31
32 // Parâmetros Auxiliares :: Instruções do Tipo M
33 localparam LDRImediato = 2'b10;
34
35 // Parâmetros Auxiliares :: Instruções do Tipo A
36 localparam HLT = 4'b0001, IN = 4'b0010;
37
38 always @ (*)
39 begin
40 if (Inibe == 1'b0)
41 begin
42 case (Opcode[7:6])
43 TipoD: begin
44 case (Opcode[4:0])
45 ADD: begin
46 CTRLDadoReg <= 2'b01;
47 CTREscritaCPSR <= 1'b0;
48 CTRLopULA <= 4'b0000;
49 CTREscritaReg <= 1'b1;
50 end
51 ADDS: begin
52 CTRLDadoReg <= 2'b01;
53 CTREscritaCPSR <= 1'b1;

```

```

54 CTRLopULA <= 4'b0000;
55 CTREscritaReg <= 1'b1;
56 end
57 SUB: begin
58 CTRLDadoReg <= 2'b01;
59 CTREscritaCPSR <= 1'b0;
60 CTRLopULA <= 4'b0001;
61 CTREscritaReg <= 1'b1;
62 end
63 SUBS: begin
64 CTRLDadoReg <= 2'b01;
65 CTREscritaCPSR <= 1'b1;
66 CTRLopULA <= 4'b0001;
67 CTREscritaReg <= 1'b1;
68 end
69 RSB: begin
70 CTRLDadoReg <= 2'b01;
71 CTREscritaCPSR <= 1'b1;
72 CTRLopULA <= 4'b0010;
73 CTREscritaReg <= 1'b1;
74 end
75 MUL: begin
76 CTRLDadoReg <= 2'b01;
77 CTREscritaCPSR <= 1'b0;
78 CTRLopULA <= 4'b0011;
79 CTREscritaReg <= 1'b1;
80 end
81 UDIV: begin
82 CTRLDadoReg <= 2'b01;
83 CTREscritaCPSR <= 1'b0;
84 CTRLopULA <= 4'b0100;
85 CTREscritaReg <= 1'b1;
86 end
87 NOT: begin
88 CTRLDadoReg <= 2'b01;
89 CTREscritaCPSR <= 1'b0;
90 CTRLopULA <= 4'b0101;
91 CTREscritaReg <= 1'b1;
92 end
93 AND: begin
94 CTRLDadoReg <= 2'b01;
95 CTREscritaCPSR <= 1'b0;
96 CTRLopULA <= 4'b0110;
97 CTREscritaReg <= 1'b1;
98 end
99 ORR: begin
100 CTRLDadoReg <= 2'b01;
101 CTREscritaCPSR <= 1'b0;
102 CTRLopULA <= 4'b0111;
103 CTREscritaReg <= 1'b1;
104 end
105 EOR: begin
106 CTRLDadoReg <= 2'b01;

```

Unidade de Controle

```

107          CTRLEscritaCPSR<= 1'b0;
108          CTRLopULA      <= 4'b1000;
109          CTRLEscritaReg <= 1'b1;
110          end
111      MOV: begin
112          CTRLDadoReg    <= 2'b10;
113          CTRLEscritaCPSR<= 1'b0;
114          CTRLopULA      <= 4'b0000;
115          CTRLEscritaReg <= 1'b1;
116          end
117      CMP: begin
118          CTRLDadoReg    <= 2'b00;
119          CTRLEscritaCPSR<= 1'b1;
120          CTRLopULA      <= 4'b0001;
121          CTRLEscritaReg <= 1'b0;
122          end
123      default: begin
124          CTRLDadoReg    <= 2'b00;
125          CTRLEscritaCPSR<= 1'b0;
126          CTRLopULA      <= 4'b0000;
127          CTRLEscritaReg <= 1'b0;
128          end
129      endcase
130      CTRLDesvio      <= 2'b00;
131      CTRLHalt        <= 1'b0;
132      CTRLEscritaMem  <= 1'b0;
133      CTRLEndMem      <= 2'b00;
134      CTRLDadoImediato <= {1'b0, Opcode[5]};
135      end
136      TipoB: begin
137          case (Opcode[5:4])
138              B: begin
139                  CTRLDesvio      <= 2'b01;
140                  CTRLEscritaReg <= 1'b0;
141                  end
142              BX: begin
143                  CTRLDesvio      <= 2'b10;
144                  CTRLEscritaReg <= 1'b0;
145                  end
146              BL: begin
147                  CTRLDesvio      <= 2'b01;
148                  CTRLEscritaReg <= 1'b1;
149                  end
150              default: begin
151                  CTRLDesvio      <= 2'b00;
152                  CTRLEscritaReg <= 1'b0;
153                  end
154          endcase
155          CTRLHalt        <= 1'b00;
156          CTRLDadoReg      <= 2'b11;
157          CTRLEscritaMem  <= 1'b0;
158          CTRLEndMem      <= 2'b00;
159          CTRLEscritaCPSR <= 1'b0;

```

```

160          CTRLopULA      <= 4'b0000;
161          CTRLDadoImediato <= 2'b00;
162          end
163      TipoM: begin
164          CTRLDesvio      <= 2'b00;
165          CTRLHalt        <= 1'b0;
166          if (Opcode[4:3] == LDRImediato)
167              begin
168                  CTRLDadoReg    <= Opcode[4:3];
169              end
170          else
171              begin
172                  CTRLDadoReg    <= 2'b00;
173              end
174          CTRLEscritaMem  <= Opcode[5];
175          CTRLEndMem      <= Opcode[4:3];
176          CTRLEscritaCPSR <= 1'b0;
177          CTRLopULA      <= 4'b0000;
178          CTRLDadoImediato <= 2'b01;
179          CTRLEscritaReg  <= ~Opcode[5];
180          end
181      TipoA: begin
182          CTRLDesvio      <= 2'b00;
183          if (Opcode[5:2] == HLT)
184              begin
185                  CTRLHalt        <= 1'b1;
186              end
187          else
188              begin
189                  CTRLHalt        <= 1'b0;
190              end
191          CTRLDadoReg      <= 2'b10;
192          CTRLEscritaMem  <= 1'b0;
193          CTRLEndMem      <= 2'b00;
194          CTRLEscritaCPSR <= 1'b0;
195          CTRLopULA      <= 4'b0000;
196          CTRLDadoImediato <= 2'b10;
197          if (Opcode[5:2] == IN)
198              begin
199                  CTRLEscritaReg <= 1'b1;
200              end
201          else
202              begin
203                  CTRLEscritaReg <= 1'b0;
204              end
205          end
206      default: begin
207          CTRLDesvio      <= 2'b00;
208          CTRLHalt        <= 1'b0;
209          CTRLDadoReg      <= 2'b00;
210          CTRLEscritaMem  <= 1'b0;
211          CTRLEndMem      <= 2'b00;
212          CTRLEscritaCPSR <= 1'b0;

```

```

213          CTRLopULA      <= 4'b0000;
214          CTRLDadoImediato <= 2'b00;
215          CTRLEscritaReg  <= 1'b0;
216          end
217      endcase
218      end
219      else
220          begin
221              CTRLDesvio      <= 2'b00;
222              CTRLHalt        <= 1'b0;
223              CTRLDadoReg      <= 2'b00;
224              CTRLEscritaMem  <= 1'b0;
225              CTRLEndMem      <= 2'b00;
226              CTRLEscritaCPSR <= 1'b0;
227              CTRLopULA      <= 4'b0000;
228              CTRLDadoImediato <= 2'b00;
229              CTRLEscritaReg  <= 1'b0;
230          end
231      end
232  endmodule
233

```

Multiplexador

```
1  module MUX
2  (
3  // Entradas :: Dados
4  input wire [31:0] Dado1, Dado2, Dado3, Dado4,
5
6  // Entradas :: Entrada de Seleção
7  input wire [1:0] Selecao,
8
9  // Saídas :: Dado de Saída
10 output reg [31:0] DadoSaída
11 );
12
13     always @ (*)
14     begin
15         case (Selecao[1:0])
16             2'b00: DadoSaída <= Dado1;
17             2'b01: DadoSaída <= Dado2;
18             2'b10: DadoSaída <= Dado3;
19             2'b11: DadoSaída <= Dado4;
20         endcase
21     end
22
23 endmodule
```

- 4 entradas

- Seleção de 2 bits



Adder

```
1  module Adder
2  (
3  // Entradas :: Dados
4  input wire [31:0] Dado1, Dado2,
5
6  // Saída :: Soma
7  output reg [31:0] Soma
8  );
9
10     always @ (*)
11         begin
12             Soma <= Dado1 + Dado2;
13         end
14
15 endmodule
```

- Soma as duas entradas





Implementação em Verilog: Integração

```

1 module Integracao
2 (
3 // Entradas
4 input wire CLK,
5 input wire [31:0] DadoEntrada,
6
7 // Saídas
8 output wire [31:0] NovoEndereco, EnderecoAtual,
9 output wire [31:0] DadoSelecioneado, DadoRegLeitura1, DadoRegLeitura2, DadoRegEscrita,
10 SaídaULA, DadoMemoria,
11 output wire [31:0] EndSelecioneado,
12 output wire Inibe,
13 output wire [31:0] Imediato,
14 output wire [4:0] RN, RM, RD,
15 output wire [3:0] Cond, NovasFlags, FlagsNZCV
16 );
17
18 // Conexões Intermediárias
19 wire [31:0] ProximoEndereco, Instrucao;
20 wire [7:0] Opcode;
21
22 // Conexões Intermediárias :: Sinais de Controle
23 wire [3:0] CTRLopULA;
24 wire [1:0] CTRLDesvio, CTRLDadoImediato, CTRLEndMem, CTRLDadoReg;
25 wire CTRLEscritaReg, CTRLEscritaCPSR, CTRLEscritaMem, CTRLHalt;
26
27 // Parâmetros
28 localparam Zero = 32'b0, Um = 32'b00000000000000000000000000000000;
29
30 PC PC (
31 .CTRLHalt(CTRLHalt),
32 .CLK(CLK),
33 .NovoEndereco(NovoEndereco),
34 .EnderecoAtual(EnderecoAtual)
35 );
36
37 Adder ADD (
38 .Dado1(EnderecoAtual),
39 .Dado2(Um),
40 .Soma(ProximoEndereco)
41 );
42
43 MUX MUX1 (
44 .Dado1(ProximoEndereco),
45 .Dado2(Imediato),
46 .Dado3(DadoRegLeitura1),
47 .Dado4(Zero),
48 .Selecao(CTRLDesvio),
49 .DadoSaida(NovoEndereco)
50 );
51
52 MemInstrucoes MemInstrucoes (
53 .EnderecoAtual(EnderecoAtual),
54 .CLK(CLK),
55 .Instrucao(Instrucao)
56 );
57
58 UnidadeDecodificacao UnidadeDecodificacao (
59 .Instrucao(Instrucao),
60 .Cond(Cond),
61 .Opcode(Opcode),
62 .RN(RN),
63 .RM(RM),
64 .RD(RD),
65 .Imediato(Imediato)
66 );
67
68 BancoRegistradores BancoRegistradores (
69 .RN(RN),
70 .RM(RM),
71 .RD(RD),
72 .DadoRegEscrita(DadoRegEscrita),
73 .CTREscritaReg(CTREscritaReg),
74 .CLK(CLK),
75 .DadoRegLeitura1(DadoRegLeitura1),
76 .DadoRegLeitura2(DadoRegLeitura2)
77 );

```

```

77
78 MUX MUX2 (
79 .Dado1(DadoRegLeitura2),
80 .Dado2(Imediato),
81 .Dado3(DadoEntrada),
82 .Dado4(Zero),
83 .Selecao(CTRLDadoImediato),
84 .DadoSaida(DadoSelecioneado)
85 );
86
87 ULA ULA (
88 .Dado1(DadoRegLeitura1),
89 .Dado2(DadoSelecioneado),
90 .CTRLopULA(CTRLopULA),
91 .SaídaULA(SaídaULA),
92 .NovasFlags(NovasFlags)
93 );
94
95 CPSR CPSR (
96 .NovasFlags(NovasFlags),
97 .CTREscritaCPSR(CTREscritaCPSR),
98 .CLK(CLK),
99 .FlagsNZCV(FlagsNZCV)
100 );
101
102 MUX MUX3 (
103 .Dado1(Imediato),
104 .Dado2(DadoRegLeitura1),
105 .Dado3(Zero),
106 .Dado4(SaídaULA),
107 .Selecao(CTREndMem),
108 .DadoSaida(EndSelecioneado)
109 );
110
111 MemPrincipal MemPrincipal (
112 .EndLeitura(EndSelecioneado),
113 .EndEscrita(EndSelecioneado),
114 .DadoMemEscrita(DadoRegLeitura2),
115 .CTREscritaMem(CTREscritaMem),
116 .CLK(CLK),
117 .DadoMemoria(DadoMemoria)
118 );
119
120 MUX MUX4 (
121 .Dado1(DadoMemoria),
122 .Dado2(SaídaULA),
123 .Dado3(DadoSelecioneado),
124 .Dado4(ProximoEndereco),
125 .Selecao(CTRLDadoReg),
126 .DadoSaida(DadoRegEscrita)
127 );
128
129 Unidadevalidacao Unidadevalidacao (
130 .Cond(Cond),
131 .FlagsNZCV(FlagsNZCV),
132 .Inibe(Inibe)
133 );
134
135 UnidadeControle UnidadeControle (
136 .Opcode(Opcode),
137 .Inibe(Inibe),
138 .CTRLDesvio(CTRLDesvio),
139 .CTRLDadoImediato(CTRLDadoImediato),
140 .CTREndMem(CTREndMem),
141 .CTRLDadoReg(CTRLDadoReg),
142 .CTREscritaReg(CTREscritaReg),
143 .CTREscritaCPSR(CTREscritaCPSR),
144 .CTREscritaMem(CTREscritaMem),
145 .CTRLopULA(CTRLopULA),
146 .CTRLHalt(CTRLHalt),
147 );
148
149 endmodule

```

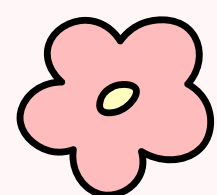
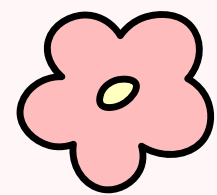


Código Teste: Cálculo de Média Aritmética

Código em ARM Assembly

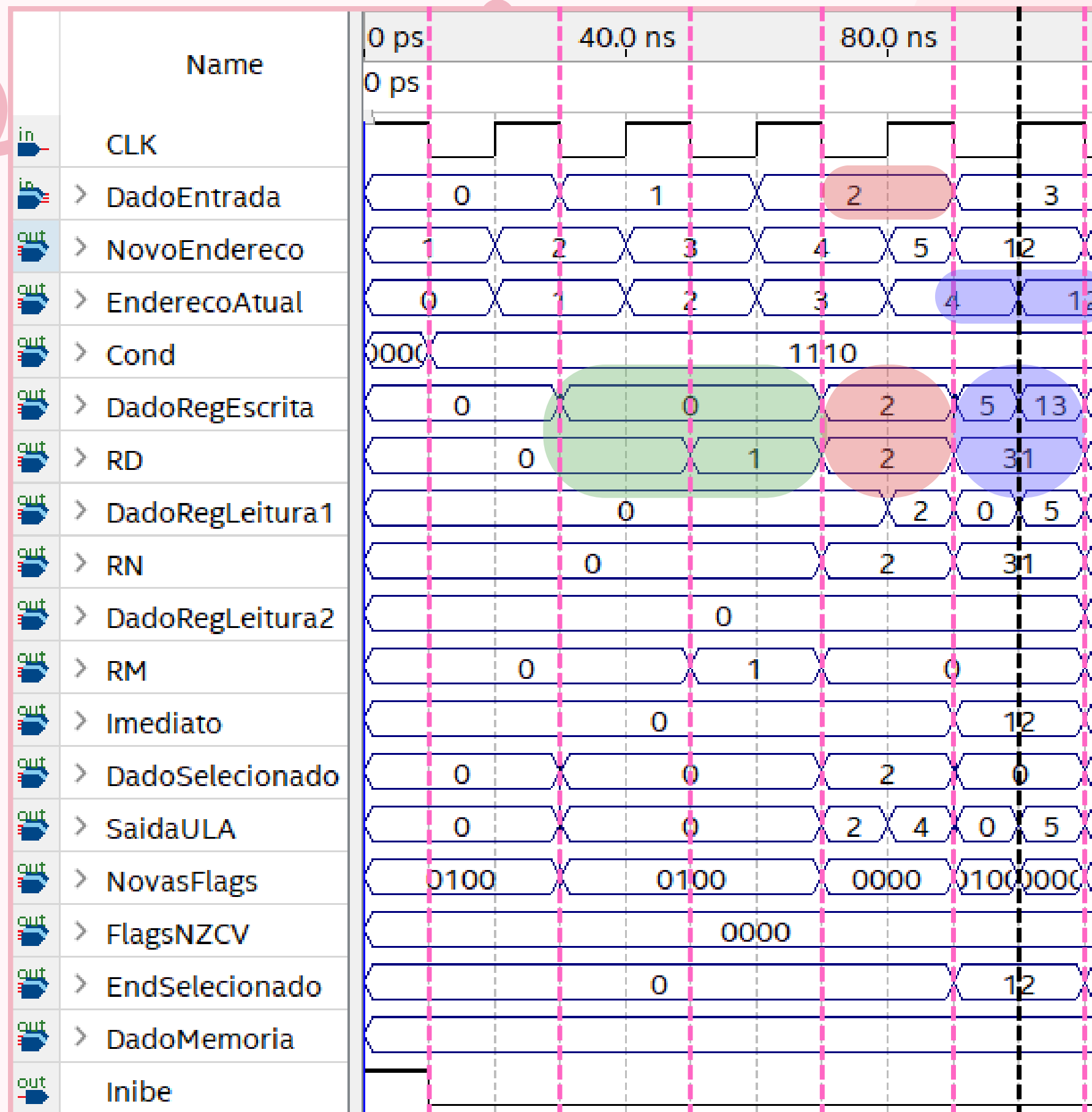


0	NOP		9	LDR	R5, =4
1	LDR	R0, #0	10	OUT	R5
2	LDR	R1, #0	11	HLT	
3	IN	R2	12	soma	CMP R1, R2
4	BL	soma	13	BXGE	R31
5	CMP	R2, #0	14	IN	R3
6	UDIVNE	R0, R0, R2	15	ADD	R0, R0, R3
7	MOV	R4, #3	16	ADD	R1, R1, #1
8	STR	R0, [R4, #1]	17	B	soma



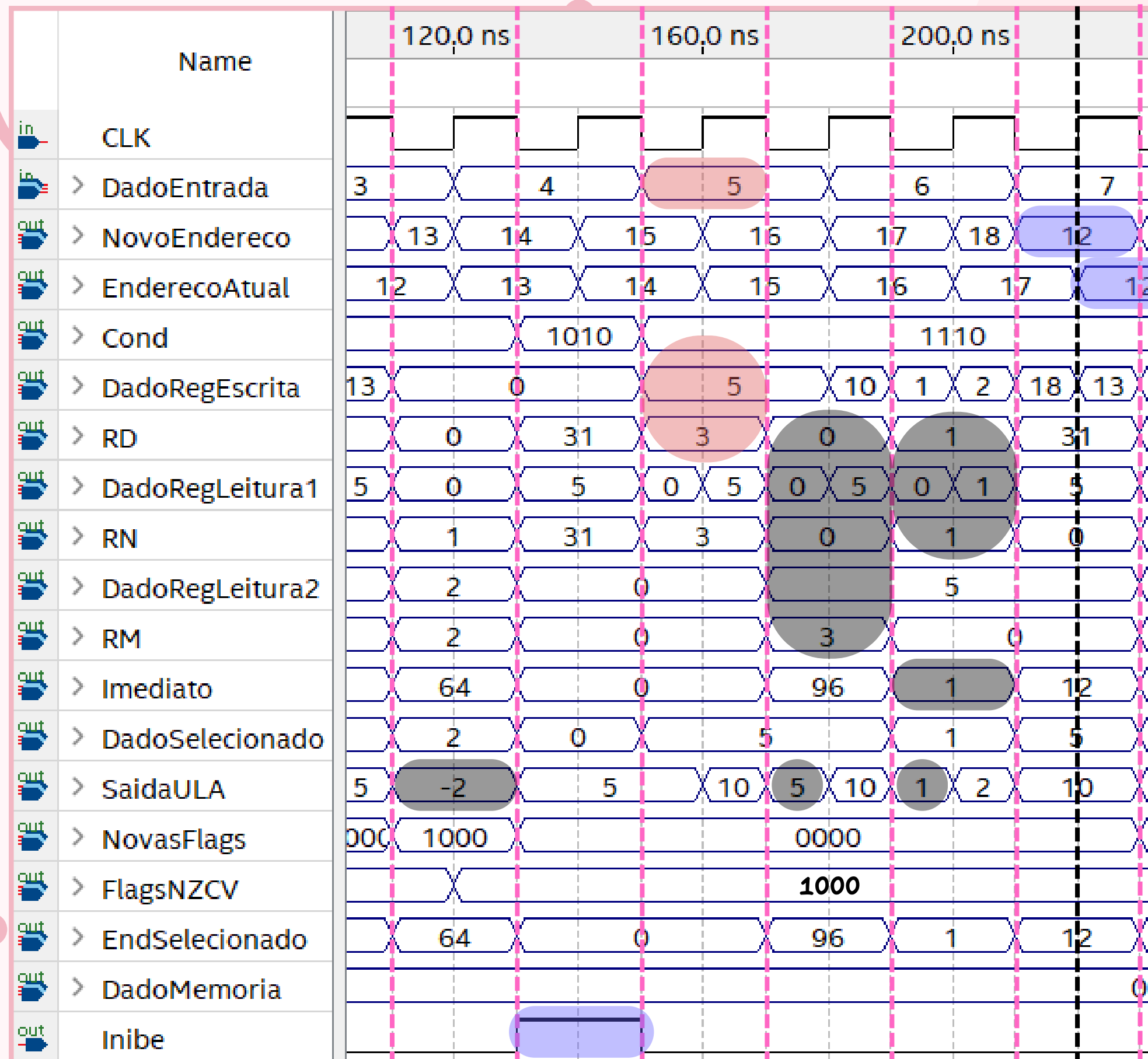


Código Teste:
Forma de Onda



0 **NOP**
 1 **LDR** **R0, #0**
 2 **LDR** **R1, #0**
 3 **IN** **R2**
 4 **BL** **Soma**

Registrador	Valor
R0	0
R1	0
R2	2



12 soma **CMP** R1, R2

13 **BXGE** R31

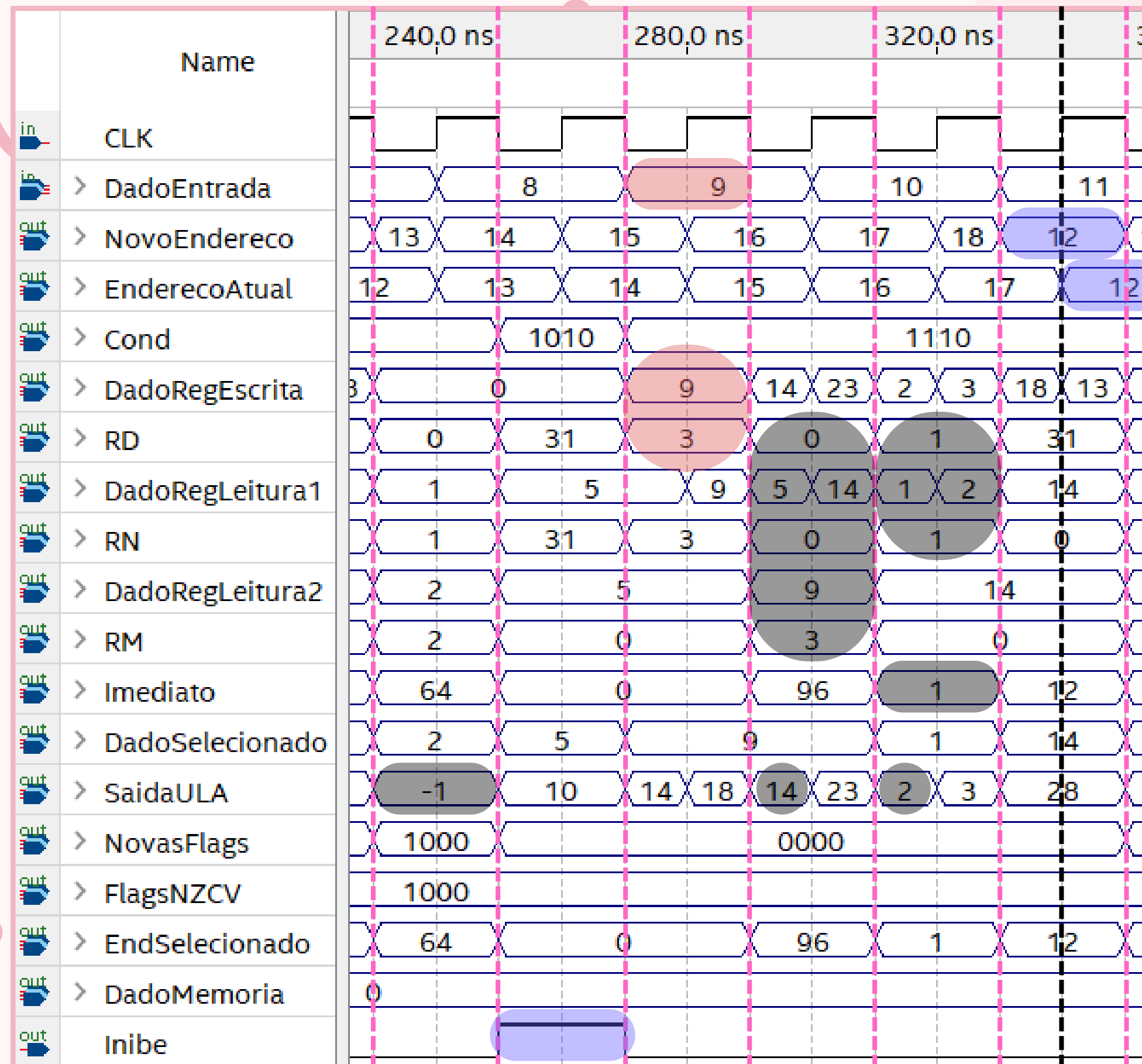
14 **IN** R3

15 **ADD** R0, R0, R3

16 **ADD** R1, R1, #1

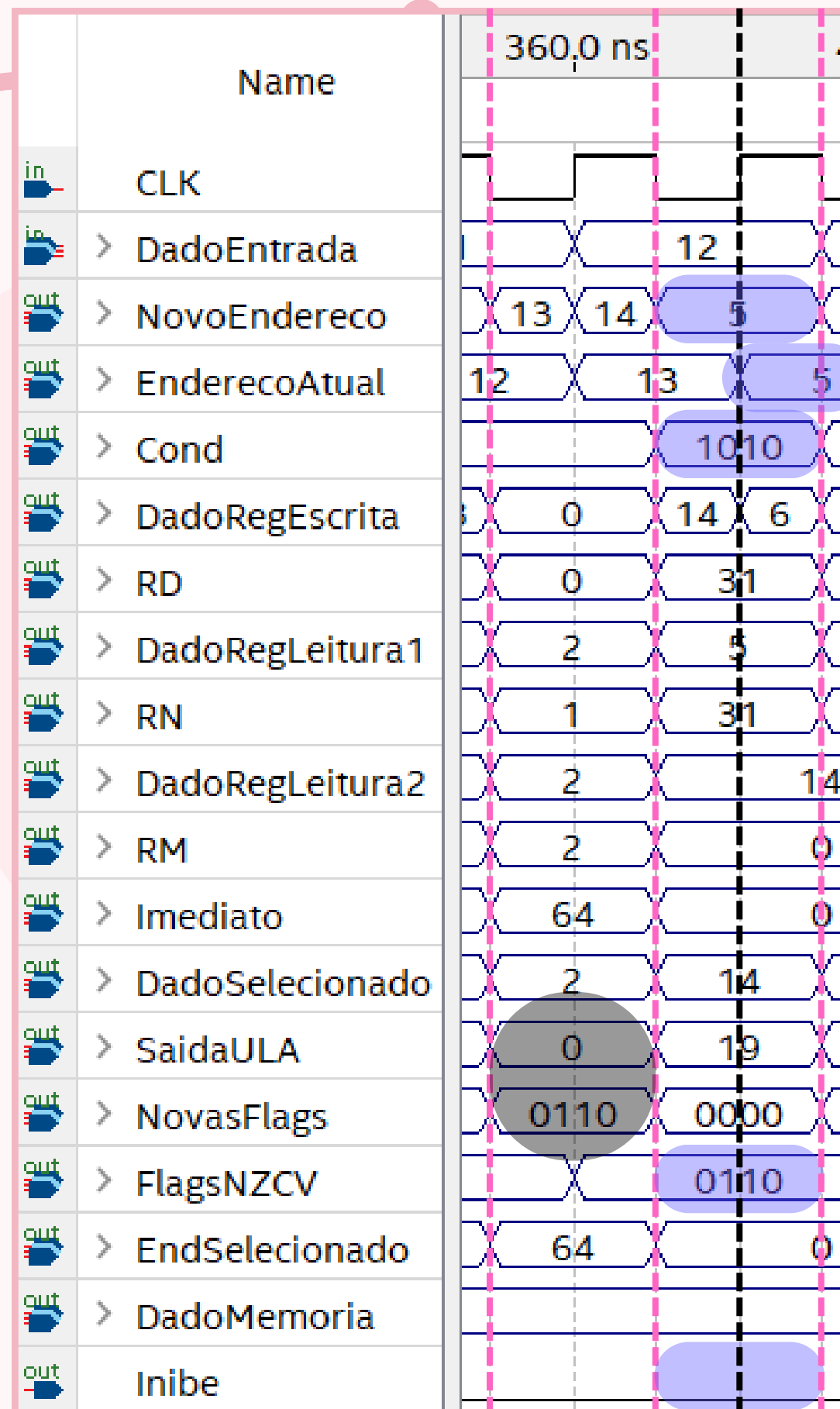
17 **B** soma

Registrador	Valor
R0	0 (5)
R1	0 (1)
R2	2



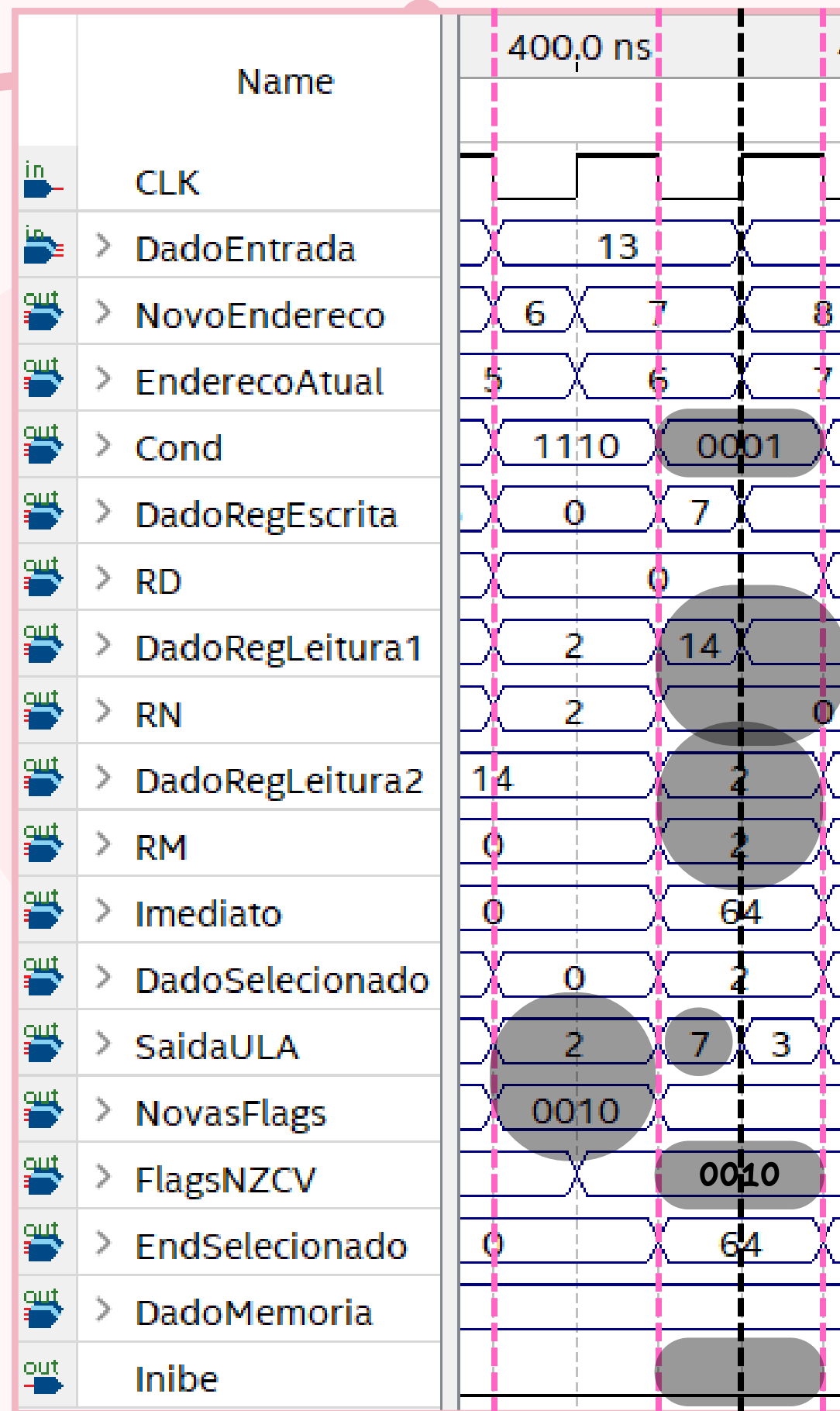
12 soma **CMP** R1, R2
 13 **BXGE** R31
 14 **IN** R3
 15 **ADD** R0, R0, R3
 16 **ADD** R1, R1, #1
 17 **B** soma

Registrador	Valor
R0	5 (14)
R1	1 (2)
R2	2



12 soma **CMP** R1, R2
 13 **BXGE** R31
 14 **IN** R3
 15 **ADD** R0, R0, R3
 16 **ADD** R1, R1, #1
 17 **B** soma

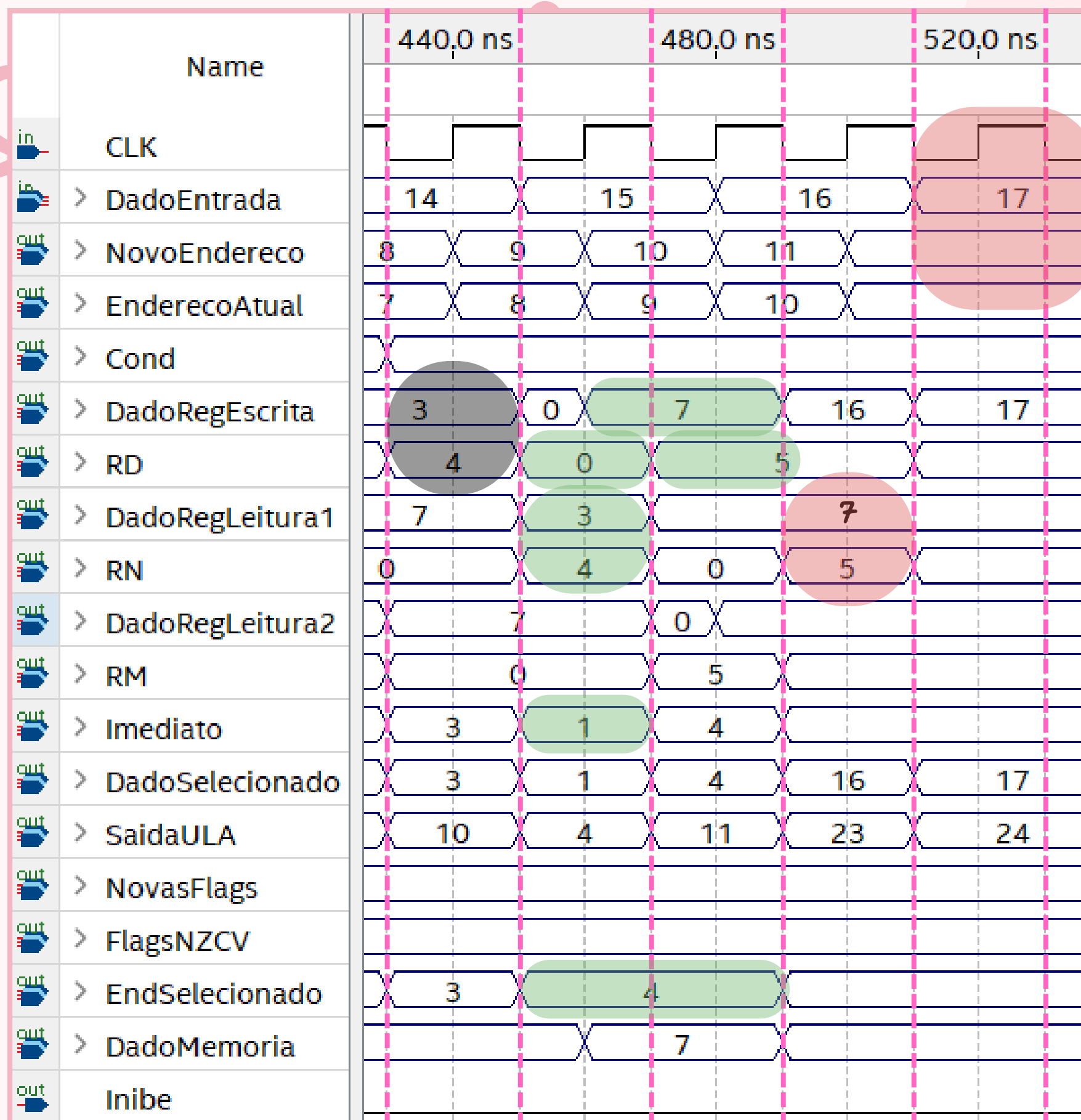
Registrador	Valor
R0	14
R1	2
R2	2



5 **CMP** **R2, #0**

6 **UDIVNE** **R0, R0, R2**

Registrador	Valor
R0	14 (7)
R1	2
R2	2



```

7  MOV    R4, #3
8  STR    R0, [R4, #1]
9  LDR    R5, =4
10 OUT    R5
11 HLT

```

Registrador	Valor
R0	7
R4	? (3)
R5	? (7)



Obrigada pela
atenção!